Vergleich von CLP(FD) in SWI-Prolog und dem CP-SAT-Solver von Google OR-Tools

Jonas Lorenz

Studiengang: Bachelor Informatik

4. September 2025

Inhaltsverzeichnis

1 Einleitung **3**

2 Grundlagen und verwandte Arbeiten **3**
2.1 Constraints und Constraint-Programmierung 3
2.2 Zentrale Mechanismen und Eigenschaften 4
2.2.1 Propagation 4
2.2.2 Backtracking 4
2.2.3 Monotonie 4
2.3 CLP(FD) in SWI-Prolog 5
2.4 Google OR-Tools und der CP-SAT-Solver 6
2.5 Verwandte Arbeiten 7

3 Vergleich von CLP(FD) und CP-SAT **7**
3.1 Interne Modellverarbeitung 7
3.2 Suchverfahren und Propagation 8
3.3 Monotonie 8
3.4 Benchmark 9
3.4.1 Rahmenbedingungen 9
3.4.2 Das N-Queens Problem 10
3.4.3 Ergebnisse und Interpretation für N-Queens 10
3.4.4 Das 7-11-Problem 11
3.4.5 Ergebnisse und Interpretation für 7-11 12

4 Einordnung und Fazit **13**

5 Anhang **14**

6 Literaturverzeichnis **16**

1 Einleitung

Constraint-Programmierung und -Solving sind zentrale Lösungsansätze für zahlreiche Kombinatorik- und Optimierungsprobleme in Industrie und Forschung. Dazu zählen beispielsweise Anwendungen im Bereich der künstlichen Intelligenz [Pop+22] sowie in der Planung industrieller Abläufe [FA97].

Anders als beim klassischen imperativen Programmieren, bei dem die Lösung eines Problems sequentiell beschrieben wird, formuliert die Constraint-Programmierung ausschließlich die Bedingungen, die eine Lösung erfüllen muss. Es handelt sich somit um einen deklarativen Ansatz; die konkrete Belegung der Variablen wird anschließend von einem Constraint-Solver ermittelt.

Das CLP(FD)-Modul in der deklarativen Sprache SWI-Prolog sowie der CP-SAT-Solver aus der Google-OR-Tools-Bibliothek sind zwei unterschiedliche Implementierungen der ganzzahligen Constraint-Programmierung. Obwohl OR-Tools eine Vielzahl weiterer – sowohl kommerzieller als auch Open-Source – Solver unterstützt, wird in dieser Arbeit CP-SAT betrachtet. CP-SAT gilt als leistungsstärkster Bestandteil der OR-Tools-Bibliothek und erzielte unter anderem bei der MiniZinc Challenge 2024 [Min24; PDG23] in allen Kategorien den ersten Platz.

Ziel der Arbeit ist es, die beiden genannten Constraint-Systeme kompakt zu präsentieren und sie hinsichtlich ihrer Such- und Propagationsverfahren, der internen Modellverarbeitung und ihres Monotonie-Verhaltens zu vergleichen. Des Weiteren erfolgt ein praktischer Vergleich der Constraint-Solver mittels eines Benchmarks unter Betrachtung des N-Queens-Problems [BS08] und des 7-11-Problems [PG83].

2 Grundlagen und verwandte Arbeiten

2.1 Constraints und Constraint-Programmierung

Das englische Wort „Constraint“ lässt sich mit Einschränkung oder Nebenbedingung übersetzen. In der Informatik beschreibt es eine Bedingung, die jede gültige Lösung eines Problems erfüllen muss. Constraints eignen sich allgemein gut, um unvollständige Informationen darzustellen. Sie ermöglichen es, die Eigenschaften und Beziehungen von Objekten zu beschreiben, auch wenn diese nur teilweise bekannt sind [FA97].

Wie bereits in Abschnitt 1 beschrieben, verfolgt die Constraint-Programmierung einen deklarativen Ansatz. Dabei wird ein Problem modelliert, indem folgende, elementare Bestandteile beschrieben werden: die Variablen, deren Wertebereiche sowie sowohl vordefinierte als auch benutzerdefinierte Constraints, die die Variablen zueinander in Beziehung setzen und den Lösungsraum einschränken.

Ein modelliertes Problem wird anschließend durch einen Constraint-Solver gelöst. Dabei handelt es sich um eine spezielle Softwarekomponente, die die Erfüllbarkeit eines Constraint-Problems prüft und konkrete Lösungen für eben dieses berechnet, sofern Lösungen existieren [Tri13; HW07].
Das Vorgehen eines klassischen Constraint-Solvers besteht aus drei grundlegenden Schritten:

1. Festlegen einer Variable durch Auswahl und Zuweisung eines Werts aus ihrem aktuellen Wertebereich,

2. Propagation der Konsequenzen dieser Festlegung auf die übrigen Variablen,

3. Fortsetzen mit der nächsten Variable oder – bei Inkonsistenz – Rücksprung zu einer vorherigen Entscheidung (Backtracking siehe 2.2.2) und Wahl einer alternativen Belegung.

Dieser Vorgang erfolgt so lange rekursiv, bis entweder eine gültige Lösung oder aber keine Lösung gefunden wurde [HW07].

2.2 Zentrale Mechanismen und Eigenschaften

In diesem Abschnitt werden zentrale Mechanismen und Eigenschaften vorgestellt, welche von Constraint-Solvern angewandt werden, um beschriebene Probleme systematisch zu lösen. Die konkrete Ausprägung von eben diesen ist abhängig vom jeweiligen Solver.

2.2.1 Propagation

Propagation bezeichnet die Ableitung logischer Konsequenzen aus bestehenden Constraints und den aktuellen Wertebereichen der Variablen [HSD92]. Es geht im Kern darum, inkonsistente Werte aus den jeweiligen Wertebereichen zu entfernen. Als „inkonsistent“ wird in diesem Kontext eine Belegung für eine Variable beschrieben, mit welcher ein Constraint (oder mehrere) nicht mehr erfüllt werden kann.

Constraint-Systeme nutzen hierzu Propagatoren, die auf Änderungen in den Wertebereichen reagieren und ggf. diese inkonsistenten Werte entfernen. Dieser Vorgang wird wiederholt, bis ein stabiler Zustand (Fixpunkt) erreicht ist, bei dem keine weitere Reduzierung möglich ist [RBW06].

2.2.2 Backtracking

Getroffene Variablenbelegungen sowie die daraus resultierenden Propagationen lassen sich durch einen Entscheidungsbaum darstellen. Stellt der Solver während der Expansion des Baumes einen Konflikt mit anderen Constraints fest, dann wird im Entscheidungsbaum zum vorherigen Entscheidungsknoten zurückgesprungen und trifft – wenn möglich – eine andere Wahl. Dieser Prozess wird als Backtracking bezeichnet [RBW06]. Sollte keine weitere Alternative zur Verfügung stehen, dann wird das Prüfen auf Alternativen und eventuelles Zurücksetzen rekursiv fortgeführt, potenziell bis zur Wurzel. Dort endet die Suche.

Wird der Entscheidungsbaum während des Backtrackings vollständig expandiert, dann gibt es keine Lösung für das modellierte Problem [HW07].

2.2.3 Monotonie

Ein Constraint-System heißt monoton, wenn das Hinzufügen weiterer Constraints keine neuen Lösungen erzeugt [Tri13; TW+25]. In nicht-monotonen Systemen kann die Reihenfolge der Constraint-Angaben zu unterschiedlichen Ergebnissen führen, wie Triska [Tri13, S. 37] anhand zweier identischer, lediglich in der Reihenfolge variierender Anfragen zeigt.

Für das Constraint-System selbst stellt nicht-monotones Verhalten kein Problem dar, es kann jedoch die Fehlersuche im Entwicklungsprozess erschweren.

Zur Lösung des Problems existieren unterschiedliche Ansätze: Eine Möglichkeit besteht darin, Monotonie syntaktisch zu erzwingen, was zwar die Modellierung etwas einschränkt, jedoch konsistente Ergebnisse gewährleistet. Alternativ setzen einige Solver-Architekturen bereits von Grund auf auf ein monotones Modell, wodurch die Eigenschaft inhärent gegeben ist. Konkrete Ausprägungen dieser Ansätze werden in Abschnitt 3.3 im Rahmen des Vergleichs erläutert.

2.3 CLP(FD) in SWI-Prolog

Die folgenden Erläuterungen stützen sich auf die Dissertation [Tri13] sowie einen Artikel [Tri12] von Markus Triska, in denen zentrale Eigenschaften und Designentscheidungen der CLP(FD)-Bibliothek dargestellt werden.

CLP(FD) – kurz für *Constraint Logic Programming over Finite Domains* – ist eine von Markus Triska im Rahmen seiner Promotion entwickelte Bibliothek für die deklarative Programmiersprache SWI-Prolog und Teil der Standardinstallation. Sie ermöglicht die Modellierung und Lösung kombinatorischer Probleme, die durch ganzzahlige Variablen mit endlichem Wertebereich beschrieben werden können.

Der allgemeine Ablauf zur Lösung eines Constraint-Problems mit CLP(FD) besteht aus drei Komponenten:

1. Deklarierung der Variablen und initiale Angabe deren Wertebereiche mit `ins/2`.
2. Definierung der Constraints durch arithmetische Operationen (z.B. `#=`, `#\=`, `#<`) oder globalen Constraints wie `all_distinct`.
3. Starten der Lösungssuche mit `label/1` oder `labeling/2`. Hierbei kommen Labeling, Propagation und Backtracking zum Einsatz.

Um das Vorgehen zu veranschaulichen folgt ein einfaches Beispiel zur Constraint-Lösung mit CLP(FD):

```
1    ?- use_module(library(clpfd)),
2       [X, Y, Z] ins 1..10,
3       X #< Y, Y #< Z,
4       X + Y + Z #= 15,
5       label([X, Y, Z]).
```

Die Zeilen zwei bis vier entsprechen den ersten beiden Schritten: Variablen deklarieren und Constraints setzen. In Zeile 5 wird mit `label/1` die Suche gestartet.

Das Suchen nach einer Lösung für das modellierte Problem wird vom Constraint-Solver von CLP(FD) durchgeführt. Dabei ist die garantierte Terminierung der Propagation eine zentrale Eigenschaft, da SWI-Prolog Variablen mit unendlichem Wertebereich unterstützt. Ohne Einschränkung könnte eine Propagation in diesem Fall nie terminieren, da eine Schranke einer unendlichen Variablen beliebig oft erhöht oder verringert werden kann. Um dies zu verhindern, dürfen die Schranken sowie deren Spanne bei einer unendlichen Variablen nur einmal verändert werden. Dadurch kann keine Endlosschleife entstehen, in

der der Solver unendlich lang die Schranken verändert, ohne Fortschritt in der Propagation zu erzielen.

Beim Lösen von Constraint-Problemen können mitunter sehr große Zwischenergebnisse auftreten, die in einigen Systemen zu Überläufen führen. CLP(FD) in SWI-Prolog unterstützt daher beliebig große Ganzzahlen und kann mit unendlichen Wertebereichen arbeiten. Dies erweitert den Anwendungsbereich, da sich auch Probleme mit natürlich großen oder theoretisch unendlichen Domänen modellieren lassen, ohne auf spezielle Arithmetik ausweichen zu müssen. Insbesondere auf 32-bit-Systemen ist diese Eigenschaft von Vorteil, da dort Überläufe deutlich schneller auftreten als auf moderneren 64-bit-Systemen.

Während die garantierte Terminierung eine Eigenschaft des Constraint-Solvers ist, betreffen die Unterstützung beliebig großer Ganzzahlen sowie die Möglichkeit Monotonie syntaktisch zu erzwingen die gesamte CLP(FD)-Bibliothek.

2.4 Google OR-Tools und der CP-SAT-Solver

Die Google OR-Tools-Bibliothek, „OR“ kurz für *Operations Research*, ist in C++ implementiert und bietet offizielle Schnittstellen unter anderem für Python und Java. In dieser Arbeit wird sie ausschließlich über Python verwendet. Zur Verwendung muss die Bibliothek zunächst installiert werden (beispielsweise über den Python-Paketmanager `pip`).

Constraint-Probleme können in OR-Tools allgemein über ganzzahlige Variablen mit endlichen Wertebereichen modelliert werden. Der grundlegende Modellierungsablauf ist analog zu CLP(FD):

1. Deklaration der Variablen und ihrer Wertebereiche mit `NewIntVar`.
2. Festlegen der Constraints: allgemeine boolesche Constraints über `Add`, globale Constraints wie `AllDifferent` über spezialisierte Methoden.
3. Starten der Suche mit `CpSolver().Solve(model)`.

Zur Demonstration folgt dasselbe Modell wie in Abschnitt 2.3:

```
1  from ortools.sat.python import cp_model as cp
2  m = cp.CpModel()
3  X = m.NewIntVar(1, 10, "X")
4  Y = m.NewIntVar(1, 10, "Y")
5  Z = m.NewIntVar(1, 10, "Z")
6  m.Add(X < Y)
7  m.Add(Y < Z)
8  m.Add(X + Y + Z == 15)
9  s = cp.CpSolver()
10 s.Solve(m)
```

Die grundlegende Architektur des Solvers ist ein hybrider Portfolio-Ansatz, das heißt es werden mehrere unterschiedliche Heuristiken sowie Suchverfahren parallel mittels mehrerer Threads ausgeführt [PDG23]. Dazu gehören klassische Constraint-Propagation, Techniken aus dem SAT-Solving und Methoden der linearen Optimierung [Kru25]. Während

der Laufzeit tauschen die Threads (In OR-Tools-Terminologie „workers“) kontinuierlich gewonnene Informationen – dazu gehören beispielsweise engere Wertebereichsgrenzen oder gefundene Teillösungen – aus. Heuristiken sind dabei zum Beispiel das regelmäßige Neustarten der Suche, der Abbruch nach einer gewissen Anzahl an Konflikten oder zeitliche Begrenzungen.

2.5 Verwandte Arbeiten

Marten [Mar18] vergleicht drei verschiedene Arten von Constraint-Solvern – CSP-, SAT- und BDD-basierte Ansätze – anhand künstlich erzeugter Modelle. Für die Tests werden die Modellparameter systematisch variiert, so zum Beispiel die Anzahl der Variablen, deren Struktur und die Anzahl zusätzlicher Nebenbedingungen. Die Ergebnisse werden anschließend mit realen Beispielen validiert. Die Arbeit zeigt, dass BDD-Ansätze bei der Zählung möglicher Konfigurationen oft am effizientesten sind, während CSP- und SAT-Solver je nach Modellstruktur ähnliche Leistungen erbringen.

In Bauer [Bau19] werden sechs gängige Constraint-Solver im Rahmen der Variabilitätsanalyse von Feature-Modellen verglichen. Dafür werden künstliche und reale Modelle kombiniert, um sowohl die Rechenzeit als auch die Vielfalt (Repräsentativität) der gefundenen Lösungen zu bewerten. Aus den Ergebnissen leitet die Arbeit praxisorientierte Empfehlungen ab, welcher Solver – je nach Anwendungsschwerpunkt – am besten geeignet ist.

Triska [Tri13] präsentiert neben den theoretischen Grundlagen und Eigenschaften von CLP(FD) auch Benchmark-Ergebnisse von Neng-Fa Zhou, dem Entwickler von B-Prolog, für verschiedene Prolog-Systeme. Das genaue Testdesign dieser Messungen wird nicht beschrieben. Die Resultate zeigen, dass CLP(FD) in Kombination mit SWI-Prolog im Vergleich eine geringere Ausführungsgeschwindigkeit aufweist und daher für viele industrielle Anwendungen weniger geeignet ist. Aufgrund seiner Eigenschaften wird es jedoch weiterhin für Lehr- und Forschungszwecke eingesetzt. Zudem ist CLP(FD) der einzige getestete Solver, der beliebig große Ganzzahlen unterstützt.

3 Vergleich von CLP(FD) und CP-SAT

3.1 Interne Modellverarbeitung

In CLP(FD) werden Variablen als *attributed variables* repräsentiert. Diese enthalten den aktuellen Wertebereich, der intern als Intervallbaum gespeichert wird. Damit lassen sich Schnitt-, Such- und Vereinigungsoperationen auf Wertebereichen effizienter durchführen. Zusätzlich enthält eine attributed variable die zugehörigen Propagatoren, die bei Änderungen des Wertebereichs aktiviert werden. Falls der Wertebereich unendlich ist wird außerdem markiert, ob eine Schranke oder die Spanne des Bereichs verändert wurde. CLP(FD) behält dabei die ursprünglichen Wertebereiche bei: Die Modellierungsebene und die interne Repräsentation stimmen also weitgehend überein [Tri13]. Die Wertebereiche der Variablen werden schon vor der eigentlichen Lösungssuche so weit wie möglich eingeschränkt.

CP-SAT hingegen kombiniert Constraint-Programmierung mit Techniken des SAT-Solvings. Dabei wird das Problem so repräsentiert, dass sich Variablen, Wertebereiche

und Constraints bei Bedarf als SAT-Klauseln erklären lassen. Anstelle einer vollständigen Vortransformation werden mithilfe von *Lazy Clause Generation* (LCG) nur dann Klauseln erzeugt, wenn sie zur Begründung von Propagation oder zur Analyse von Konflikten benötigt werden [Stu10]. Globale Constraints bleiben als spezialisierte Propagatoren erhalten und interagieren mit dem SAT-Solver, indem sie bei Bedarf Klauseln erzeugen, die die jeweilige Inferenz oder den Konflikt erklären und damit zukünftige Konflikte ausschließen [Kru25].

Zudem ist CP-SAT nicht nur auf ganzzahlige Techniken beschränkt, denn der Solver verwendet unter anderem auch Linear Relaxation. Das bedeutet, die Bedingung der Ganzzahligkeit wird temporär gelockert, um eventuelle bessere Grenzen schneller zu finden [Kru25].

Bei CP-SAT wird das Modell vor der Lösungssuche so weit wie möglich vereinfacht und eingeschränkt. Dafür werden beispielsweise Variablenbelegungen vorher erprobt, um weitere Einschränkungen zu finden. Gefundene Lösungen werden schließlich wieder zurücktransformiert.

3.2 Suchverfahren und Propagation

CLP(FD) in SWI-Prolog verwendet ein eher „klassisches“ Suchverfahren, bestehend aus Labeling, Propagation und Backtracking. Variablen werden in einer vom Anwender wählbaren oder durch Heuristiken bestimmten Reihenfolge belegt (Labeling). Jede Belegung wird unmittelbar durch Constraint-Propagation überprüft. Tritt eine Inkonsistenz auf, greift der Solver auf Backtracking zurück, um alternative Belegungen zu prüfen. Die Suche erfolgt grundsätzlich in einem einzelnen Thread.

Der CP-SAT-Solver verfolgt hingegen einen hybriden Portfolio-Ansatz. Mehrere Suchstrategien und Heuristiken werden parallel in separaten Threads (workers) ausgeführt [PDG23]. Neben klassischer Constraint-Propagation kommen Techniken aus dem SAT-Solving zum Einsatz, darunter Konfliktanalyse mit *Conflict-Driven Clause Learning* (CDCL) und Lazy Clause Generation. Diese Verfahren ermöglichen es, aus Konflikten während der Suche neue Klauseln abzuleiten, die den Suchraum dauerhaft einschränken. Informationen wie engere Wertebereichsgrenzen oder Teillösungen werden zwischen den Threads kontinuierlich ausgetauscht, wodurch unterschiedliche Suchstränge voneinander profitieren können.

Während CLP(FD) inkrementell arbeitet und jede Entscheidung unmittelbar propagiert, formuliert CP-SAT das Modell kontinuierlich in eine SAT-Form um und lernt aus Konflikten.

3.3 Monotonie

In SWI-Prolog CLP(FD) kann Monotonie über die Prolog-Flag `clpfd_monotonic` aktiviert werden. In diesem Modus ist der Operator `?/1` verpflichtend, um Variablen explizit als ganze Zahlen zu kennzeichnen. Zusammengesetzte arithmetische Terme (z. B. `(1+1)`) werden abgelehnt. Auch ist `X #=3` dann nicht mehr zulässig, weil zu wenig über die Variable `X` bekannt ist. Dadurch können durch unterschiedliche Auswertungsreihenfolgen keine neuen Lösungen entstehen [TW+25].

CP-SAT bietet keine explizite Einstellung für Monotonie. Durch CDCL und Lazy Clause Generation ergibt sich jedoch implizit monotones Verhalten im semantischen Sinn: Einmal gelernte Klauseln sind logische Konsequenzen der ursprünglichen Constraints und bleiben dauerhaft bestehen. Damit wird der Lösungsraum ausschließlich eingeschränkt. Zwar beeinflussen Heuristiken und die parallele Suche in mehreren Threads den Suchverlauf und die Laufzeit, sie können jedoch keine neuen Lösungen erzeugen oder bereits ausgeschlossene Lösungen wieder zulassen.

3.4 Benchmark

Alle Benchmarkergebnisse, weiterführende Diagramme, eine kompakte Dokumentation sowie jeglicher Quellcode sind auch auf GitHub zu finden.[1] Zusammengefasste Ergebnisse sind in Tabelle 1 und Tabelle 2 angegeben. Die modellierten Probleme für CP-SAT sind unter Programmcode 1 und Programmcode 3, für CLP(FD) unter Programmcode 2 und Programmcode 4, angegeben.

3.4.1 Rahmenbedingungen

Die Benchmarks wurden mittels eines Ryzen 7 5700X Prozessors (8 Kerne; 16 Threads; Basistakt 3,80GHz) durchgeführt. Es standen 32 GB DDR4-3666 MHz RAM zur Verfügung. Das zugrunde liegende Betriebssystem war Windows 11 Pro (x64) (Version 23H2) unter der Energieoption „Höchstleistung“. Verwendet wurde Python 3.11, OR-Tools 9.12.4544 und SWI-Prolog 9.2.9.

Die Benchmarks erfolgten in drei Varianten, wobei CLP(FD) stets mit First-Fail und ohne weitere Konfiguration arbeitete. Erste Versuche mit alternativen Auswahlstrategien führten bei wachsender Problemgröße zu langen Laufzeiten, die den Zeitrahmen des Benchmarks überschritten hätten. Um reproduzierbare Ergebnisse in vertretbarer Zeit zu gewährleisten, wurde daher First-Fail als Auswahlstrategie festgelegt.
Die folgenden Varianten wurden getestet:

- *Variante 1*: CP-SAT mit FF und single-threaded,
- *Variante 2*: CP-SAT ohne vorgegebene Strategie und single-threaded,
- *Variante 3*: CP-SAT ohne vorgegebene Strategie und multi-threaded.

Da First-Fail bei CP-SAT nicht existiert, wurde die Strategie explizit nachgebildet. Dafür wurde `model.AddDecisionStrategy()` kombiniert mit `CHOOSE_MIN_DOMAIN_SIZE` (kleinster Wertebereich) und `SELECT_MIN_VALUE` (kleinster Wert) verwendet. Die konkrete Implementierung ist unter Programmcode 5 angegeben. Wichtig zu erwähnen ist jedoch, dass sich CP-SAT durch Neustarts oder Lernmechanismen dennoch abweichend verhalten kann.

Der Parameter `Seed` wurde konstant gehalten, um die Reproduzierbarkeit der Ergebnisse zu gewährleisten. Ohne die Festlegung eines Seeds würde CP-SAT bei jedem neuen Anlauf zum Beispiel andere Heuristik-Parameter verwenden.

[1] Siehe `https://github.com/J0ne28/Vergleich-CLP-FD-und-CP-SAT`.

Gemessen wurde die tatsächlich belegte CPU-Zeit mittels `time.process_time()` in Python und `statistics(cputime, T0)` in SWI-Prolog. Schedulingeffekte oder I/O sind also nicht in die Messung eingeflossen. Die kleinste Messeinheit betrug $\frac{1}{64}$s.

Das 7-11-Problem wurde 100-mal wiederholt, das N-Queens-Problem jeweils für jedes $N \in \{25, 26, \ldots, 50\}$ 100-mal. Vor Beginn der Messungen wurden zusätzlich 5 Warmup-Läufe durchgeführt. Für das N-Queens-Problem wurde für jedes N das arithmetische Mittel über alle Durchläufe gebildet.

3.4.2 Das N-Queens Problem

Das N-Queens-Problem wurde zum ersten Mal 1848 in der Berliner Schachzeitung beschrieben. Ursprünglich mit genau acht Damen angegeben, wird es gegenwärtig auf N Damen, die auf einem $N \times N$ großen Schachbrett platziert werden, generalisiert. Die Herausforderung des Problems besteht darin, dass die Damen sich nicht gegenseitig schlagen dürfen [BS08].

Aufgrund seiner einfachen Modellierung und starkem Wachstum der Lösungsmenge in Abhängigkeit von N eignet sich das N-Queens-Problem als Beispielproblem für diesen Benchmark. Das N-Queens-Problem wird in [PF24] für CP-SAT oder in [Tri13] für CLP(FD) beispielhaft aufgeführt.

Der Fokus des N-Queens-Problems liegt bei diesem Benchmark auf der Skalierbarkeit mit wachsendem N.

3.4.3 Ergebnisse und Interpretation für N-Queens

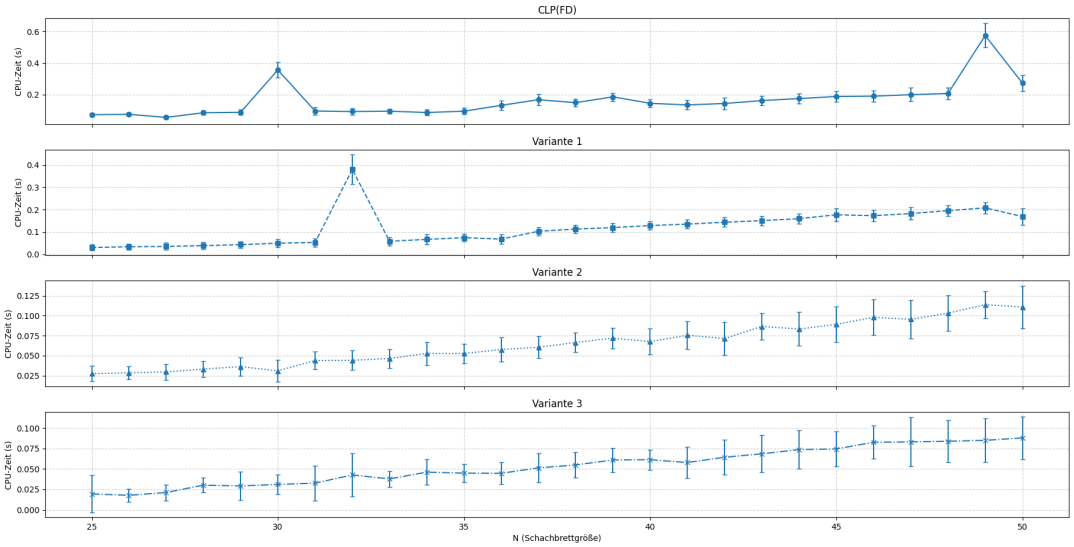


Abbildung 1: N-Queens Benchmark – Mittelwerte (100 Läufe) mit Standardabweichung.

Abbildung 1 verdeutlicht die Variation der Laufzeiten je nach Variante. Die exakten Mittelwerte und Standardabweichungen sind in Tabelle 1 dokumentiert.

Variante 1. In dieser Konfiguration zeigt sich, dass sowohl CLP(FD) als auch CP-SAT Laufzeitschwankungen aufweisen können. CLP(FD) produziert markante Peaks bei $N = 30$ und $N = 49$, die auf die statische First-Fail-Heuristik zurückzuführen sind: trifft diese eine ungünstige Variablenwahl, so muss ein größerer Teil des Suchraums expandiert werden, als bei einer „guten“ Variablenwahl. Auch CP-SAT ist in dieser Variante nicht völlig robust gegenüber diesem Effekt und zeigt bei $N = 32$ einen signifikanten Laufzeitpeak. Abgesehen davon bleibt CP-SAT aber insgesamt auf einem schnelleren Niveau als CLP(FD).

Variante 2. Ohne die explizite Vorgabe einer Heuristik erzielt CP-SAT die besten Ergebnisse im Single-Thread-Betrieb. Die Laufzeiten steigen mit wachsendem N zwar erwartungsgemäß an, Peaks wie in Variante 1 sind jedoch geglättet. Es zeigt sich, dass die dynamische Funktionsweise von CP-SAT besser skaliert als das eher statische Vorgehen von CLP(FD). CLP(FD) ist hier im Mittel langsamer und erreicht weniger stabile Laufzeiten.

Variante 3. In der multithreaded-Ausführung verstärkt sich der Vorteil von CP-SAT nochmals. Durch parallele Suchstränge und den Austausch von Informationen zwischen den Threads (z. B. bessere Schranken oder gefundene Teillösungen) werden ungünstige Heuristikentscheidungen oftmals kompensiert. Die Laufzeiten verlaufen sehr gleichmäßig, Peaks wie bei CLP(FD) treten praktisch nicht auf. CLP(FD) bleibt dagegen auch hier anfällig für starke Schwankungen und verliert deutlich an Effizienz bei einzelnen Problemgrößen.

Über alle Varianten hinweg zeigt sich, dass CP-SAT im N-Queens-Benchmark die robusteren und durchgängig schnelleren Ergebnisse liefert. CLP(FD) kann in einzelnen Fällen erheblich langsamer sein, da die Suche stark von der gewählten Heuristik abhängt. CP-SAT profitiert besonders von seinen eingebauten Restart-Mechanismen und vom Multithreading, wodurch die Lösungssuche stabilisiert und die Laufzeit im Vergleich zu CLP(FD) signifikant reduziert wird.

3.4.4 Das 7-11-Problem

Das 7-11-Problem wurde ursprünglich in [PG83] von Paul Pritchard und David Gries beschrieben. Es geht darum, vier Artikelpreise aus dem Sortiment eines Lebensmittelgeschäfts zu bestimmen, deren Summe 711 Cent beträgt. Zusätzlich muss das Produkt der Artikel $711 \cdot 100^3$ ergeben. Ein 4-Tupel, das beide Bedingungen erfüllt, stellt eine Lösung dar.

Das 7-11-Problem eignet sich als Benchmark, weil es trotz seiner geringen Variablenanzahl eine interessante Kombination von Constraints aufweist. Einerseits ist die Modellierung mit nur vier Variablen vergleichsweise einfach, andererseits erzwingt die gleichzeitige Erfüllung von Summen- und Produktbedingungen eine starke Einschränkung des Suchraums. Damit stellt das Problem hohe Anforderungen an die Constraint-Propagation.

7-11 wird in dieser Arbeit eingesetzt, um den Umgang der Solver mit komplexen arithmetischen Abhängigkeiten zu untersuchen.

3.4.5 Ergebnisse und Interpretation für 7-11

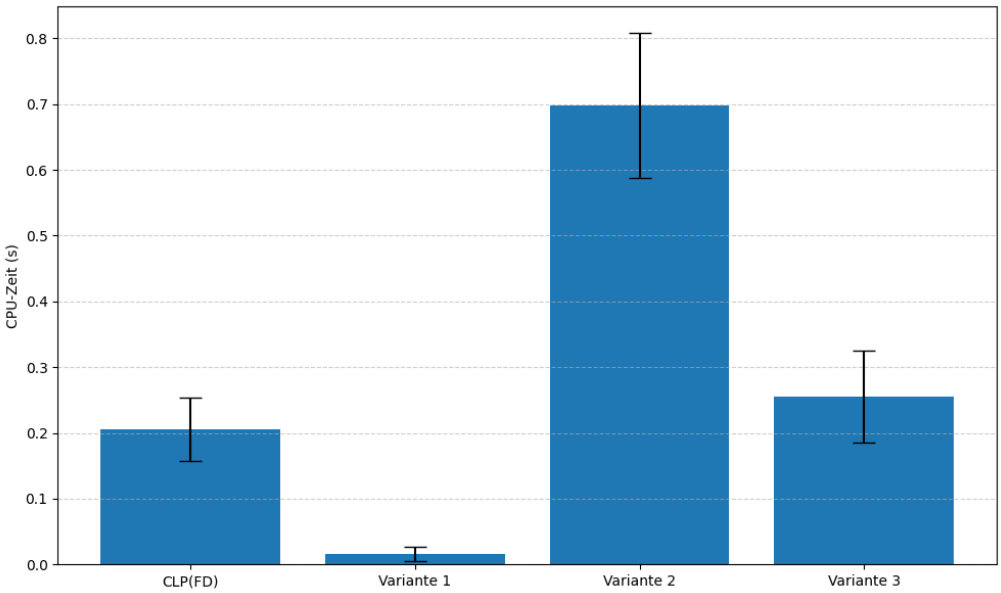


Abbildung 2: 711 Benchmark – Mittelwerte (100 Läufe) mit Standardabweichung.

Abbildung 2 visualisiert die mittleren Laufzeiten mit Streuung; die zugrunde liegenden Kennzahlen (Mittelwert, Standardabweichung) sind in Tabelle 2 ausgewiesen und bilden die Grundlage der folgenden Interpretation.

Variante 1. Mit erzwungener First-Fail-Heuristik liefert CP-SAT im 7-11-Benchmark die durchgängig besten Ergebnisse. Die Laufzeiten liegen konstant im Bereich weniger Millisekunden und sind damit schneller als CLP(FD). Die feste Variablenwahlstrategie harmoniert offenbar bei diesem stark eingeschränkten Problem sehr gut mit der Konfliktanalyse des Solvers, sodass Lösungen ohne praktisch signifikanten Suchaufwand gefunden werden konnten.

Variante 2. Ohne explizite Vorgabe einer Heuristik ist CP-SAT hier deutlich langsamer als in Variante 1. Die Laufzeit liegt deutlich über der von CLP(FD) und zeigt eine starke Schwankung. Bei einem kleinen, stark arithmetisch eingeschränkten Modell wie 7-11 und der limitierten Thread-Anzahl greifen die automatisch gewählten Heuristiken weniger effizient, sodass vergleichsweise viel Suchraum exploriert wird.

Variante 3. Die multithreaded-Ausführung reduziert die mittlere Laufzeit im Vergleich zu Variante 2, bleibt aber weiterhin über CLP(FD). Während einzelne Läufe schnell abgeschlossen werden, treten auch sporadische Ausreißer auf. Der Portfolio-Ansatz gleicht ungünstige Entscheidungen teilweise aus, bringt bei so kleinen Instanzen jedoch keinen durchgängig stabilen Vorteil.

Beim 7-11-Problem hängt die Performance von CP-SAT stark von der Konfiguration ab: Mit First-Fail (Variante 1) ist der Solver überlegen und schlägt CLP(FD). Ohne Strategie

(Variante 2) ist er dagegen langsamer als CLP(FD), die Multithread-Variante (Variante 3) verbessert die Laufzeit etwas. CLP(FD) liefert stabile Resultate, erreicht aber nicht die Leistung eines, wie in Variante 1, gut konfigurierten CP-SAT.

4 Einordnung und Fazit

CLP(FD) ermöglicht eine vergleichsweise einfache Modellierung und legt großen Wert auf formale Korrektheit. Dadurch eignet es sich vor allem für den Einsatz in der Lehre und wird dort auch aktiv verwendet [Tri13]. Da CLP(FD) nativ in SWI-Prolog implementiert und dort unmittelbar nutzbar ist, lässt sich die Bibliothek flexibel anpassen und in bestehende Prolog-Programme integrieren. Die eingesetzten Techniken und Verfahren basieren überwiegend auf den klassischen Ansätzen des Constraint-Solvings.

CP-SAT beziehungsweise OR-Tools erfordert demgegenüber eine eher ungewohnte Modellierung, da deklarative Konzepte in einer imperativen Sprache umgesetzt werden. Diese Einschätzung hängt jedoch vom Nutzer ab. Außerdem wurde in dieser Arbeit nur CP-SAT in Kombination mit Python betrachtet, die Erfahrung kann also je nach gewählter Sprache variieren. Der Solver bietet eine größere Zahl an Konfigurationsmöglichkeiten, kombiniert verschiedene Verfahren und unterstützt Multithreading.

Die durchgeführten Benchmarks erlauben eine erste vergleichende Einschätzung der Solver. In den Benchmarks zeigte sich, dass CP-SAT in vielen Fällen schnellere und stabilere Ergebnisse liefert, während CLP(FD) trotz seines geringeren Funktionsumfangs in einzelnen Szenarien vergleichbare oder sogar bessere Ergebnisse erreichen kann. Für eine umfassendere Beurteilung könnten jedoch weitere Aspekte berücksichtigt werden, etwa das Speicherverhalten oder die Stabilität bei sehr großen Instanzen. Ebenso wäre es interessant, die Systeme anhand praxisnäherer Anwendungsprobleme – beispielsweise aus dem Bereich Scheduling oder Routing – zu testen. Solche Erweiterungen würden eine noch differenziertere Bewertung ermöglichen, gehen jedoch über den Rahmen dieser Seminararbeit hinaus.

Insgesamt wird deutlich, dass die Eignung eines Solvers vom jeweiligen Kontext abhängt: CLP(FD) ist für transparente Modellierung und Lehre gut geeignet, während CP-SAT in Anwendungsszenarien mit höheren Anforderungen an Leistung und Flexibilität Vorteile aufweist. Ein naheliegender nächster Schritt bestünde darin, praxisnahe Scheduling- oder Routingprobleme zu benchmarken, um die Unterschiede der Solver auch in realitätsnahen Anwendungsszenarien zu untersuchen.

5 Anhang

N	CLP(FD) $\bar{x} \pm \sigma$ (s)	Variante 1 $\bar{x} \pm \sigma$ (s)	Variante 2 $\bar{x} \pm \sigma$ (s)	Variante 3 $\bar{x} \pm \sigma$ (s)
25	0.069 ± 0.014	0.030 ± 0.013	0.028 ± 0.010	0.020 ± 0.023
26	0.072 ± 0.014	0.034 ± 0.014	0.029 ± 0.008	0.018 ± 0.008
27	0.055 ± 0.013	0.035 ± 0.016	0.030 ± 0.010	0.021 ± 0.010
28	0.080 ± 0.016	0.039 ± 0.016	0.033 ± 0.010	0.030 ± 0.009
29	0.090 ± 0.018	0.043 ± 0.014	0.036 ± 0.011	0.029 ± 0.018
30	0.352 ± 0.052	0.050 ± 0.018	0.031 ± 0.013	0.031 ± 0.012
31	0.102 ± 0.021	0.053 ± 0.019	0.044 ± 0.011	0.033 ± 0.022
32	0.095 ± 0.020	0.380 ± 0.067	0.044 ± 0.012	0.043 ± 0.027
33	0.098 ± 0.014	0.058 ± 0.019	0.047 ± 0.012	0.038 ± 0.010
34	0.091 ± 0.017	0.067 ± 0.022	0.053 ± 0.015	0.046 ± 0.016
35	0.099 ± 0.015	0.075 ± 0.017	0.053 ± 0.012	0.045 ± 0.011
36	0.147 ± 0.019	0.068 ± 0.022	0.058 ± 0.015	0.045 ± 0.013
37	0.177 ± 0.030	0.103 ± 0.019	0.060 ± 0.014	0.051 ± 0.018
38	0.150 ± 0.025	0.113 ± 0.018	0.067 ± 0.012	0.055 ± 0.015
39	0.191 ± 0.024	0.120 ± 0.020	0.072 ± 0.013	0.061 ± 0.015
40	0.154 ± 0.017	0.129 ± 0.018	0.068 ± 0.016	0.061 ± 0.012
41	0.125 ± 0.033	0.135 ± 0.020	0.076 ± 0.017	0.058 ± 0.019
42	0.154 ± 0.039	0.144 ± 0.021	0.071 ± 0.021	0.064 ± 0.022
43	0.149 ± 0.038	0.151 ± 0.022	0.087 ± 0.017	0.068 ± 0.023
44	0.162 ± 0.040	0.160 ± 0.022	0.083 ± 0.021	0.074 ± 0.024
45	0.174 ± 0.045	0.177 ± 0.029	0.089 ± 0.022	0.074 ± 0.021
46	0.172 ± 0.046	0.173 ± 0.026	0.098 ± 0.023	0.083 ± 0.020
47	0.196 ± 0.042	0.182 ± 0.028	0.095 ± 0.024	0.083 ± 0.030
48	0.196 ± 0.044	0.196 ± 0.024	0.103 ± 0.022	0.084 ± 0.026
49	0.504 ± 0.090	0.208 ± 0.025	0.114 ± 0.017	0.085 ± 0.027
50	0.244 ± 0.055	0.169 ± 0.038	0.111 ± 0.027	0.088 ± 0.026

Tabelle 1: Laufzeiten pro N (Mittelwert $\bar{x}$ und Standardabweichung σ) für CLP(FD) und CP-SAT-Varianten.

CLP(FD) $\bar{x} \pm \sigma$ (s)	Variante 1 $\bar{x} \pm \sigma$ (s)	Variante 2 $\bar{x} \pm \sigma$ (s)	Variante 3 $\bar{x} \pm \sigma$ (s)
0.206 ± 0.049	0.016 ± 0.011	0.698 ± 0.111	0.255 ± 0.071

Tabelle 2: Laufzeiten für das 7-11-Problem (Mittelwert $\bar{x}$ und Standardabweichung σ) für CLP(FD) und CP-SAT-Varianten.

```
1 from ortools.sat.python import cp_model
2
3 def n_queens_cpsat(n: int) -> bool:
4     model = cp_model.CpModel()
```

```
    Q  = [model.NewIntVar(0, n-1, f"q_{i}")      for i in range(n)]
    d1 = [model.NewIntVar(0, 2*n-2, f"d1_{i}")   for i in range(n)]
    d2 = [model.NewIntVar(0, 2*n-2, f"d2_{i}")   for i in range(n)]
    model.AddAllDifferent(Q)
    for i in range(n):
        model.Add(d1[i] == Q[i] + i)
        model.Add(d2[i] == Q[i] - i + (n-1))
    model.AddAllDifferent(d1)
    model.AddAllDifferent(d2)
    solver = cp_model.CpSolver()
    solver.parameters.random_seed = 12345
    return solver.Solve(model) in (cp_model.OPTIMAL, cp_model.FEASIBLE
    ↪ )
```

Programmcode 1: N-Queens in CP-SAT – Modellierung und Lösung

```
:- use_module(library(clpfd)).

queens(N, Qs) :-
    length(Qs, N),
    Qs ins 1..N,
    all_distinct(Qs),
    safe_diagonals(Qs),
    labeling([ff], Qs).

safe_diagonals(Qs) :-
    length(Qs, N),
    numlist(1, N, Is),
    maplist(sum_index,  Qs, Is, D1),
    maplist(diff_index, Qs, Is, D2),
    all_distinct(D1),
    all_distinct(D2).

sum_index(Q, I, S)  :- S #= Q + I.
diff_index(Q, I, D) :- D #= Q - I.
```

Programmcode 2: N-Queens in CLP(FD) – Modellierung und Lösung

```
from ortools.sat.python import cp_model

def seven_eleven():
    m = cp_model.CpModel()
    max_price = 711
    p = [m.NewIntVar(1, max_price, f"p{i}") for i in range(4)]
    m.Add(sum(p) == 711)
    target = 711 * 100**3
    prod = m.NewIntVar(1, target, "prod")
    m.AddMultiplicationEquality(prod, p)
    m.Add(prod == target)
    s = cp_model.CpSolver()
```

```
13     s.parameters.random_seed = 12345
14     status = s.Solve(m)
15     if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
16         return [s.Value(x) for x in p]
17     return None
```

Programmcode 3: 7–11 in CP-SAT – Modellierung und Lösung

```
1 :- use_module(library(clpfd)).
2
3 seven_eleven(Vars) :-
4     Vars = [A,B,C,D],
5     Vars ins 1..711,
6     A + B + C + D #= 711,
7     A * B * C * D #= 711*100^3,
8     labeling([ff], Vars).
```

Programmcode 4: 7–11 in CLP(FD) – Modellierung und Lösung

```
1 from ortools.sat.python import cp_model
2
3 model = cp_model.CpModel()
4 model.AddDecisionStrategy(*Liste der Variablen*,  cp_model.
    ↪ CHOOSE_MIN_DOMAIN_SIZE, cp_model.SELECT_MIN_VALUE)
```

Programmcode 5: First-Fail-Nachbildung in Python

6 Literaturverzeichnis

[Bau19] Martin Bauer. „A Comparison of Six Constraint Solvers for Variability Analysis“. Masterarbeit. Universität Passau, Fakultät für Informatik und Mathematik, Sep. 2019.

[BS08] Jordan Bell und Brett Stevens. „A survey of known results and research areas for *n*-queens“. In: *Discrete Mathematics* 309.1 (2008), S. 1–31.

[FA97] Thom Frühwirth und Slim Abdennadher. *Constraint-Programmierung: Grundlagen und Anwendungen*. Springer-Verlag Berlin Heidelberg, 1997.

[HSD92] Pascal Van Hentenryck, Helmut Simonis und Mehmet Dincbas. „Constraint satisfaction using constraint logic programming“. In: *Artificial Intelligence* 58.1-3 (1992), S. 113–159.

[HW07] Petra Hofstedt und Armin Wolf. *Einführung in die Constraint-Programmierung*. Springer-Verlag Berlin Heidelberg, 2007.

[Kru25] Dominik Krupke. *The CP-SAT Primer: Using and Understanding Google OR-Tools' CP-SAT Solver*. Technische Universität Braunschweig. 2025. URL: https://d-krupke.github.io/cpsat-primer/00_intro.html (besucht am 04. 07. 2025).

[Mar18] Adrian Marten. „A Comparison Study of Domain Constraint Solver for Model Counting“. Masterarbeit. Universität Passau, Fakultät für Informatik und Mathematik, März 2018.

[Min24] MiniZinc. *MiniZinc Challenge 2024*. Abgerufen am 12. August 2025; enthält offizielle Regeln, Teilnahmedaten und Ergebnisstrukturen. 2024. URL: https://www.minizinc.org/challenge/2024/ (besucht am 12. 08. 2025).

[PDG23] Laurent Perron, Frédéric Didier und Steven Gay. „The CP-SAT-LP Solver“. In: *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*. Bd. 280. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 3:1–3:2. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CP.2023.3.

[PF24] Laurent Perron und Vincent Furnon. *OR-Tools*. Version v9.8. Google, 7. Mai 2024. URL: https://developers.google.com/optimization/.

[Pop+22] Andrei Popescu u. a. „An overview of machine learning techniques in constraint solving“. In: *Journal of Intelligent Information Systems* 58.1 (2022), S. 91–118.

[PG83] Paul Pritchard und David Gries. *The Seven-Eleven Problem*. Techn. Ber. TR 83-574. Department of Computer Science, Cornell University, 1983.

[RBW06] Francesca Rossi, Peter van Beek und Toby Walsh. *Handbook of Constraint Programming*. Elsevier Science, 2006.

[Stu10] Peter J. Stuckey. „Lazy Clause Generation: Combining the Power of SAT and CP (and MIP?) Solving“. In: *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*. Hrsg. von Andrea Lodi, Michela Milano und Paolo Toth. Berlin, Heidelberg: Springer, 2010, S. 5–9.

[Tri12] Markus Triska. „The Finite Domain Constraint Solver of SWI-Prolog“. In: Mai 2012, S. 307–316.

[Tri13] Markus Triska. „Correctness Considerations in CLP(FD) Systems“. Dissertation. Wien: Technische Universität Wien, 2013.

[TW+25] Markus Triska, Jan Wielemaker u. a. *library(clpfd): Constraint Logic Programming over Finite Domains*. Online documentation, zuletzt abgerufen am 11.08.2025. SWI-Prolog. 2025. URL: https://www.swi-prolog.org/pldoc/man?section=clpfd.